TCP: Connection Management

Lecture 18

Computer Networks and Distributed Programming

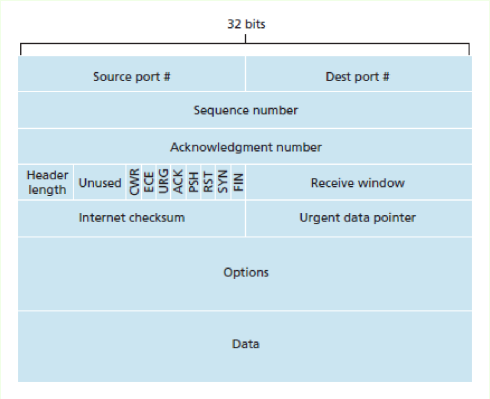
Where we are: the numbers have to start somewhere

Everything so far assumed the connection was already there: buffers, variables, two numbers counting bytes.

- Before any of it, the two ends must agree that they are talking, and on the number each will count from.
- The same problem returns at the end: stopping without leaving one side talking to a peer that has gone.
- Nothing new is needed in the header — three flag bits do all of it, and we have never used them.
- Of the four things transport does, this is the one your own code has been asking for all along: `connect`, `accept`, `close`.

The three bits we have not used

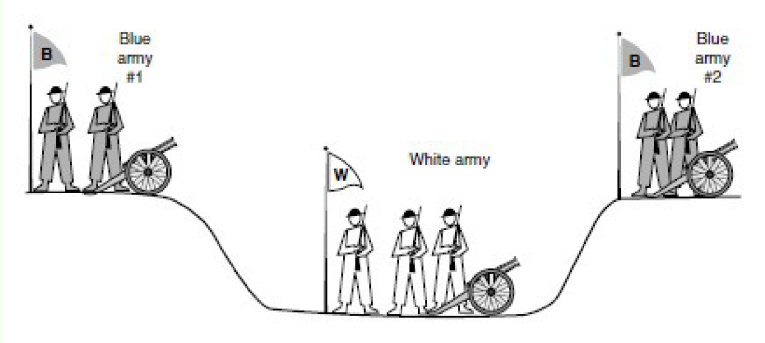
- SYN: this segment opens a connection. It is set in the first two segments of a connection and never again.
- FIN: the sender has no more data. It closes one direction, and is sent once per direction.
- RST: there is no connection here. Throw the segment away and do not send it again.
- The sequence and acknowledgement fields carry the agreement; these three bits say what kind of agreement it is.



Two hosts must agree, and twice

- TCP is connection-oriented: the state is in the two end systems, and it has to be created and destroyed on both.
- The only tool for agreeing is the network — which loses and damages the very messages that carry the agreement.
- Lose one of them and the two ends disagree: one believes the connection is open while the other has already released it.
- So nothing counts until it has been acknowledged, and the opening is built out of that rule.

The two-army problem

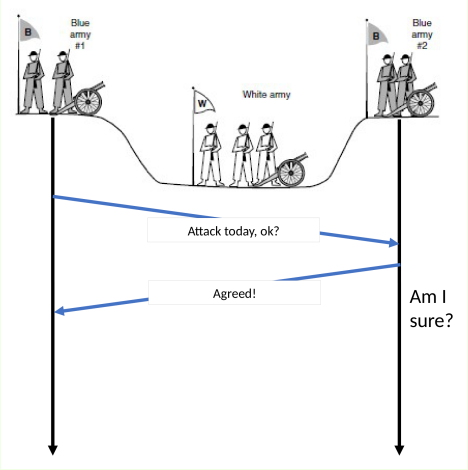


A white army camps in the valley. Two blue armies hold the hillsides: either one alone loses, together they win — but only if they attack at the same moment. Every messenger they send crosses the valley and may be caught.

Is there a protocol that lets the blue armies win?

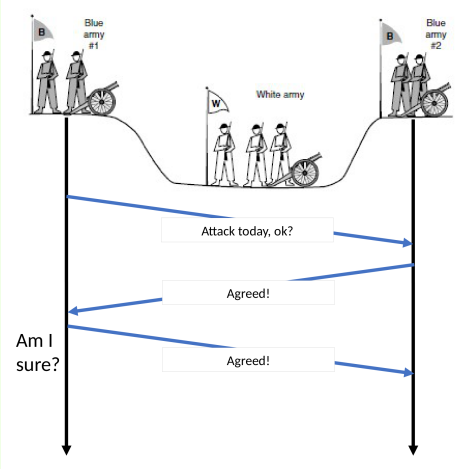
Two messages are not enough

- Blue #1 proposes: *attack today*. The messenger gets through, #2 agrees, and the reply also gets through.
- Nobody attacks. Commander #2 does not know that his reply arrived, and knows #1 will not attack alone.
- The uncertainty is not about the message: it is about what the other knows.
- Two messages leave the second sender in exactly the position the first one was in.



Three messages are not enough either

- Add an acknowledgement of the reply. Now #2 is sure — and #1 is not: did *his* acknowledgement arrive?
- A fourth message moves the doubt back to #2, a fifth back to #1, and so on without end.
- It can be proven that no protocol exists: over a channel that loses messages, common certainty is unreachable.
- The three-way handshake is not perfect. It is adequate, which is a different claim and the one TCP makes.



What a handshake can and cannot buy

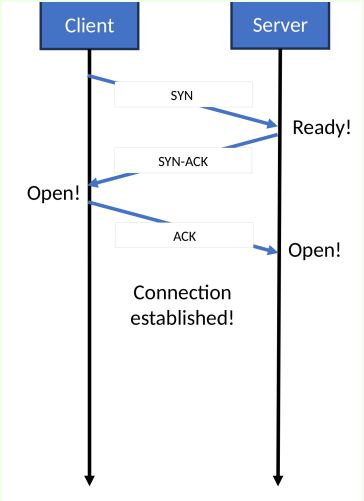
What is impossible

That both ends be certain, at the same instant, that the other is ready. No number of messages achieves it.

- What TCP needs is weaker: each end has learnt the other's initial number, and knows its own was received.
- Three segments deliver exactly that, and nothing in them has to be believed on faith — each is confirmed by the next.
- The doubt that remains is the sender's about his last message, and the first data segment settles it.
- Where that doubt does cost something is at the close, and we come back to it with the timers.

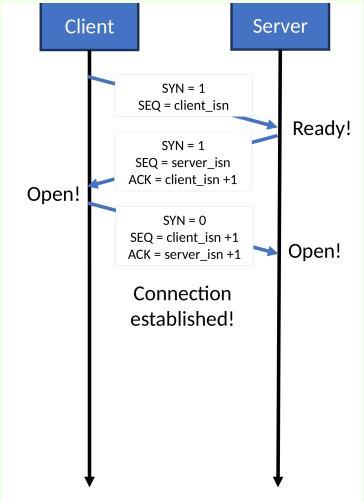
Three segments before anything is said

- The client's `connect` sends a connection request, a segment with the SYN bit set and no data.
- The server answers with a SYN of its own that also acknowledges the client's: one segment, SYNACK.
- The client acknowledges that, and this third segment may already carry application data.
- One full round trip, plus whatever the server takes to turn around, before the first byte of the request.



The three segments, field by field

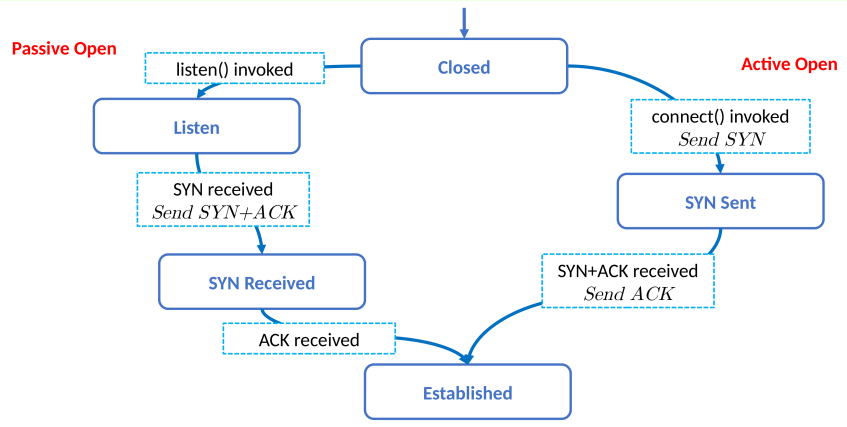
- SYN: `SYN=1`, no payload, `seq = client_isn`, chosen at random by the client.
- SYNACK: `SYN=1`, `ACK=1`, `seq = server_isn`, `ack = client_isn+1`. The server allocates buffers and variables here.
- ACK: `SYN=0`, `ack = server_isn+1`. The client has now allocated its own.
- A SYN carries no data but consumes one sequence number — which is why every acknowledgement here is `isn+1`.



Why the initial number is drawn at random

- Nothing forces a connection to start at zero, and starting there would be the worst choice available.
- A segment from an earlier connection between the same four numbers may still be wandering in the network.
- With a fresh starting point it falls outside the window of the new connection and is discarded instead of accepted as data.
- An attacker who can guess the numbers can inject segments into a connection he cannot see: randomising them is a defence.
- And the +1 coming back is a small proof of its own: only somebody who received the SYN could have written it.

Passive open, active open



Two paths through one machine. The server waits — passive open — and answers a SYN with a SYNACK; the client asks — active open — and sends the SYN itself. They meet at `ESTABLISHED`, the server one segment later than the client.

The same states, from inside your code

- `listen` *is* the transition into `LISTEN`. From that instant the kernel answers SYNs by itself, with no process involved.
- `connect` sends the SYN and blocks; it returns when the SYNACK has arrived and the third segment has been sent.
- `accept` takes no part in the handshake at all: it hands you a connection that is already established.
- So a server can be slow to call `accept` without any client noticing — the connection is open, and the data waits in a buffer.
- The handshake is done by the two kernels. Your program reads the result, and its only report is a descriptor or an error.

What `listen` really allocates

Two queues, not one

A listening socket has a queue of half-open connections — SYN received, third segment not yet — and a queue of completed ones waiting to be accepted.

- An arriving SYN creates an entry in the first queue; the final ACK moves that entry into the second.
- `accept` removes one entry from the second queue: the backlog you passed to `listen` is that queue's length.
- If it fills up, new connections are dropped or refused, and the client sees a delay without ever learning the reason.
- The first queue is smaller, older and unguarded. It is the one an attacker aims at.

The handshake costs a round trip

- Before the first byte of a request leaves, one full round trip has already been spent on agreeing to talk.
- On a local network that is nothing. Across an ocean it is a tenth of a second, per connection.
- This is exactly what persistent connections buy at the application layer: pay the handshake once, and reuse what it built.
- It costs the server too: buffers and variables are committed on the first segment, before anything has been asked of it.

Nobody answers, or somebody refuses

Your client calls `connect` and the SYN gets no answer. How long does it wait?

Tens of seconds. TCP retransmits the SYN a few times with a growing interval, then abandons the attempt and `connect` fails.

- A refusal is much faster: if the host is up but nothing is bound to that port, it replies with a segment carrying RST.
- Your program sees the two as different errors — a timeout, or the `ECONNREFUSED` you got the first time you started the client before the server.
- For UDP that report came from the network layer instead. Here the answer is a transport segment, from the peer's TCP.

The SYN flood

What if the third segment never comes?

The server has already committed buffers and variables. It holds the half-open connection for a minute or more before reclaiming it.

- Send enough SYNs and the queue of half-open connections is full of entries that will never complete.
- Legitimate clients are then refused: what has been exhausted is a queue, not bandwidth — the attack needs very little traffic.
- The source addresses are forged, so the SYNACKs go to hosts that never asked, and the attacker never has to reply.
- It was among the first documented denial-of-service attacks, and it is still one of the most common.

SYN cookies

- The defence inverts the problem: answer the SYN without remembering it.
- `server_isn` is not random but computed — a hash of the two addresses, the two ports, and a secret only the server knows.
- No state is kept, so a forged SYN costs the server one reply and nothing else at all.
- A legitimate client returns that number plus one. The server recomputes the hash, compares, and only then creates the connection and its socket.
- The queue never fills because until the third segment there is nothing in it. Most operating systems now do this.

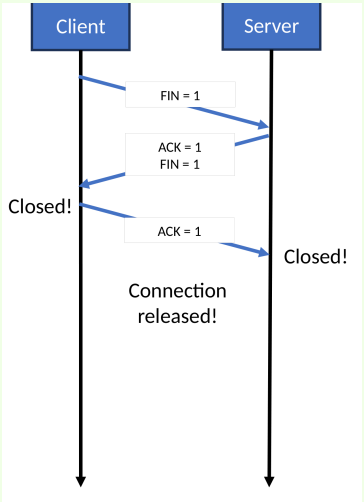
What three answers to a SYN tell you

What comes back	What it means	
SYNACK	something is listening there	open
RST	the host is up, that port is not served	closed
nothing	it never arrived: something dropped it	filtered

One SYN per port, and the three answers map a machine from outside without ever completing a connection. The usual tool for it is `nmap`, and we will use it when we come to diagnosing a network.

Closing takes two closes

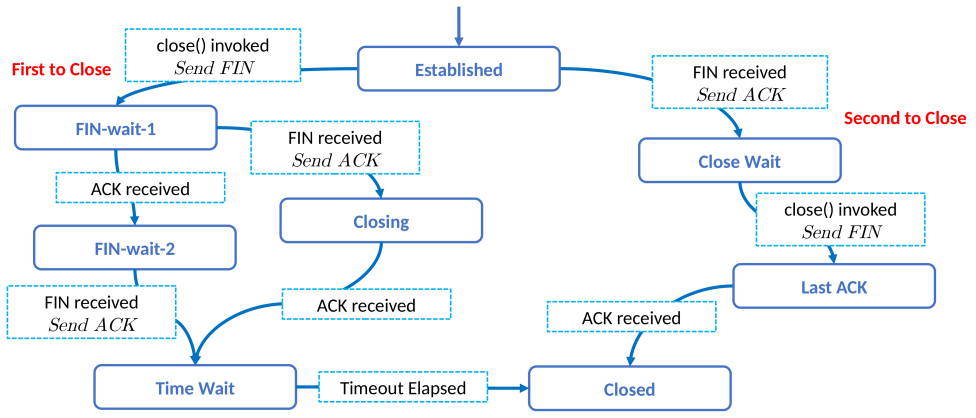
- Either end may start. Say the client does: its `close` sends a segment with the FIN bit set.
- The server acknowledges it, and when it has nothing more to send, sends a FIN of its own.
- The client acknowledges that last FIN, and the state on both sides can be released.
- The figure draws the server's two answers as one segment. That happens, but it is not the normal case.



Half-close: one direction at a time

- A FIN says *I have no more data to send*. It does not say *this connection is over*: it closes one direction.
- After sending its FIN the client can still receive; after acknowledging it the server can still send, for as long as it likes.
- So the two FINs are independent, and the normal release is four segments: FIN, ACK, then FIN, ACK the other way.
- They collapse into three only when the peer has nothing left to send and closes at once: then its ACK and its FIN travel together.
- `close` sends the FIN and gives up the descriptor. `shutdown(fd, SHUT_WR)` sends it and keeps reading — which is how a client says *that was the whole request*.

The states of a release



Whoever closes first walks down the left; whoever answers walks down the right, and sits in `CLOSE-WAIT` until its own application closes. The state in the middle is for the case where both send a FIN at the same time and neither is second.

The descriptor nobody closed

Your server's `recv` returns 0 and you forget to close the descriptor. What breaks?

Nothing, at first. The connection stays in `CLOSE-WAIT` for ever: the peer's FIN was acknowledged, but your FIN is never sent.

- The peer is left waiting too. The release never finishes on either side, and neither program is told.
- A few thousand clients later the server is out of descriptors and stops accepting. The failure appears far from its cause.
- `ss -tn state close-wait` lists exactly those. A column of `CLOSE-WAIT` is always a bug in the program, never in the network.
- That 0 is the FIN arriving. It is an instruction to close, not an error to ignore.

TIME-WAIT

- The end that closes first does not go straight to `CLOSED`. It waits — typically thirty seconds to two minutes.
- First reason: if its last acknowledgement is lost, the peer will resend its FIN, and somebody has to be there to answer.
- Second reason: segments of this connection may still be in the network, and its four numbers could be handed to a new one.
- The wait is about twice the longest a segment can survive, so everything belonging to the old connection has died first.
- It is the price of closing first, and it is paid by whoever closes first — which is not always the client.

Address already in use

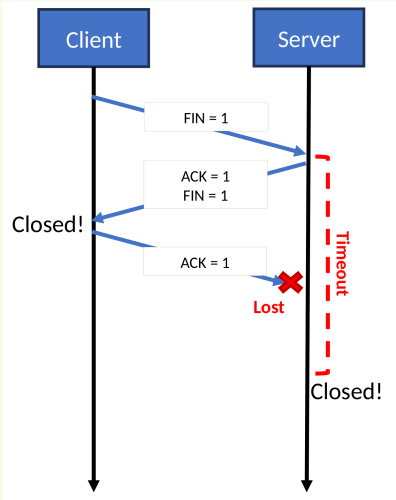
You interrupt your server, restart it, and `bind` fails. Nothing else is running. Why?

Because interrupting it made it the end that closed first. Its connections are in `TIME-WAIT`, still holding the local port, and `bind` will not take a port that a live connection is using.

- `SO_REUSEADDR` — the line you have written in every server since the first one — says bind anyway.
- It is safe: the old connection keeps its four numbers, and no segment of it can be delivered to the new socket.
- When the client closes first, which is the usual case, the server never enters `TIME-WAIT` and the problem never appears.

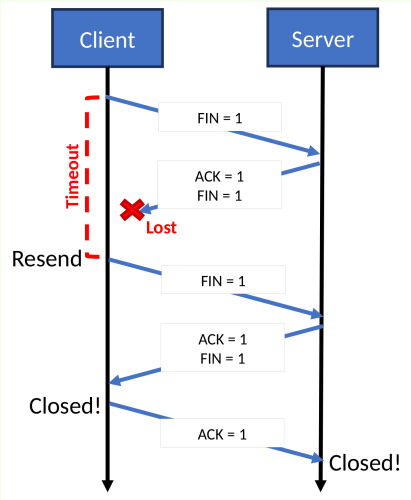
The last acknowledgement is lost

- The client sends its final ACK and considers itself closed. The segment never arrives.
- The server is left waiting for it. Its timer expires and it releases the connection anyway.
- Nothing is damaged: both ends end up closed, just not at the same instant, and neither ever finds out.
- This is the doubt of the two armies, and this is what it costs here: a delay on one side.



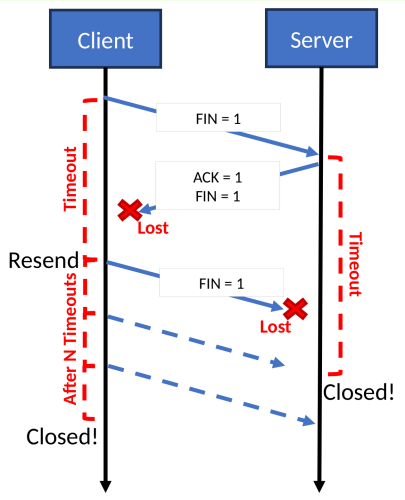
The answer to the FIN is lost

- This time it is the server's reply that is lost, so the client is still waiting to be told anything at all.
- Its timer expires and it sends the FIN again, exactly as it would retransmit a segment of data.
- The server answers the repeated FIN with the same segment it sent before, and this time it gets through.
- Retransmission at the close is not a new mechanism: a timer, and a copy kept until it is acknowledged.



When the losses keep coming

- The client resends its FIN, and the retransmission is lost too. This can go on.
- After a bounded number of attempts the client stops trying and closes unilaterally; the server's own timer closes its side.
- The connection ends with neither end having confirmed anything to the other — which is where we started this morning.
- No protocol removes that. What TCP adds is a timer, a limit on the retries, and the resources released either way.



RST: closing without asking

- A FIN ends a connection in order. An RST ends it now, and whatever was still in the buffers is discarded.
- It is sent whenever a segment arrives for a connection that does not exist: no listener on that port, or a peer that has rebooted.
- A machine that crashes sends nothing. Its peer keeps a connection that no longer exists at the other end, and finds out only when it writes.
- Then the RST comes back, and your program learns it as `ECONNRESET` on a read, or `EPIPE` and `SIGPIPE` on a write.
- The kernel's own idle probe starts after about two hours — which is why a program that must notice a dead peer sends its own heartbeat.

Two terminals, one connection

One machine, two windows. `nc` gives us a TCP endpoint with no program to write: listening on one side, connecting on the other.

```
$ nc -l 5001                    server:  listen on port 5001
Maradona
sfogliatella
```

```
$ ss -tln 'sport = :5001'       is anybody listening?
$ nc localhost 5001             client:  connect and type
Maradona
sfogliatella
```

What is typed on the client appears on the server. Back the other way travels only an acknowledgement, a segment with no data at all.

Filtering the capture

```
tcp && tcp.port==5001 && ip
```

- Three clauses, one per layer of the question: the transport protocol, the port, and the network protocol underneath.
- The middle one would be enough by itself — nothing else on this machine uses 5001. The other two make the intent explicit.
- Everything happens over 127.0.0.1: the two ends are the same host, and not one packet reaches a cable.
- So the interface to capture on is the loopback one. Point the sniffer at the real one and you will see nothing.

What the capture shows

```
$ tshark -i lo -f "tcp port 5001"
 1   0.000000  41230 -> 5001   [SYN]       Seq=0   Win=65495  Len=0
 2   0.000018   5001 -> 41230  [SYN, ACK]  Seq=0   Ack=1      Len=0
 3   0.000027  41230 -> 5001   [ACK]       Seq=1   Ack=1      Len=0
 4   6.213004  41230 -> 5001   [PSH, ACK]  Seq=1   Ack=1      Len=9
 5   6.213021   5001 -> 41230  [ACK]       Seq=1   Ack=10     Len=0
 6   9.402771  41230 -> 5001   [PSH, ACK]  Seq=10  Ack=1      Len=13
 7   9.402788   5001 -> 41230  [ACK]       Seq=1   Ack=23     Len=0
 8  19.884113  41230 -> 5001   [FIN, ACK]  Seq=23  Ack=1      Len=0
```

Three segments before a key is pressed. Then one segment per line typed, nine and thirteen bytes — the newline is in there — each with a bare acknowledgement coming back. The last line is `Ctrl-C`.

The same connection, carrying a file

```
$ nc -l 12338 > real2.doc          server:  write what arrives

$ nc 127.0.0.1 12338 < real.doc    client:  send a file
^C
```

- The two redirections do all of it: `nc` sends its standard input and prints what it receives. The copy is made over TCP.
- What travels is a byte stream with no structure: nothing on the wire says where the file ends.
- `nc` does not stop when the input runs out, so the transfer is ended by hand — and the `Ctrl-C` is what sends the FIN.
- Here the FIN is the end-of-file marker. A protocol that needs to send two files over one connection must say so itself.

Summary

- Agreement over a channel that loses messages cannot be made certain. Three segments are not a proof; they are enough.
- The handshake trades two random initial numbers and commits state on both sides: one round trip before any data moves.
- A close is two independent halves, a FIN each way, each acknowledged. `recv` returning 0 is the peer's FIN.
- `TIME-WAIT` protects the last acknowledgement and the four numbers; a `CLOSE-WAIT` that never ends is your own bug.
- `RST` is the refusal — *no such connection here* — and timers close what agreement cannot.
- Still open: the receive window in the header, and what a sender does when the limit is not the receiver but the network.